

DOKUMENTASI SESI 2

Nama : Vivi Restu Anggraini
NPM 23312144

```
    setEmail(initialEmail);  
  }  
}, [initialEmail]);  
  
const { width } = useWindowDimensions();  
const isDesktop = width > 768;  
  
const handleLogin = async () => {  
  if (isLoading) return;  
  
  if (!email || !password) {  
    Alert.alert('Error', 'Email dan Password tidak boleh kosong!');  
    return;  
  }  
  
  setIsLoading(true);  
  
  try {  
    const response = await fetch(`${API_URL}/auth/login`, {  
      method: 'POST',  
      headers: {  
        'Content-Type': 'application/json',  
      },  
      body: JSON.stringify({ email, password }),  
    });  
  });  
}
```

1. Pengembangan komponen otentikasi visual pada halaman Halaman Login (Login Page UI). Antarmuka dirancang menggunakan elemen form terkontrol Tailwind CSS untuk menangkap input pengelola. Halaman ini bertugas mengirimkan kredensial operator ke endpoint backend, mengekstrak token JWT secara asinkronus jika login sukses, dan memicu pengalihan hak akses masuk (*routing gateway*) ke dalam sistem internal dasbor utama.

```

emen-Informasi-terintegrasi-Lahan-Pertanian-Cerdas > User > src > screens > DashboardScreen.tsx > Dashb
{
  /* CARD 1: KELEMBABAN TANAH */
  <View style={styles.card, { width: isDesktop ? '23%' : '100%' }}>
    <View style={styles.cardHeader}>
      <Text style={styles.cardEmoji}>💧</Text>
      <Text style={styles.cardLabel}>KELEMBABAN TANAH</Text>
    </View>
    <Text style={styles.cardValue}>{metrics.soilMoisture}</Text>
    <Text style={styles.statusBadge, styles.badgeSuccess}>Kondisi Lahan</Text>
  </View>

  /* CARD 2: SUHU LINGKUNGAN */
  <View style={styles.card, { width: isDesktop ? '23%' : '100%' }}>
    <View style={styles.cardHeader}>
      <Text style={styles.cardEmoji}>☀️</Text>
      <Text style={styles.cardLabel}>SUHU LINGKUNGAN</Text>
    </View>
    <Text style={styles.cardValue}>{metrics.temperature}</Text>
    <Text style={styles.statusBadge, styles.badgeSuccess}>Normal</Text>
  </View>

  /* CARD 3: KELEMBABAN UDARA */
  <View style={styles.card, { width: isDesktop ? '23%' : '100%' }}>
    <View style={styles.cardHeader}>
      <Text style={styles.cardEmoji}>☁️</Text>
      <Text style={styles.cardLabel}>KELEMBABAN UDARA</Text>
    </View>
    <Text style={styles.cardValue}>{metrics.humidity}</Text>
    <Text style={styles.statusBadge, styles.badgeSuccess}>Ideal</Text>
  </View>

  /* CARD 4: KESEHATAN VEGETASI */
  <View style={styles.card, { width: isDesktop ? '23%' : '100%' }}>
    <View style={styles.cardHeader}>
      <Text style={styles.cardEmoji}>🌱</Text>
      <Text style={styles.cardLabel}>KESEHATAN VEGETASI</Text>
    </View>
  </View>
}

```

2. Perancangan antarmuka utama ringkasan performa pada modul Dashboard Overview. Komponen ini berfungsi sebagai halaman pendaratan utama (*landing dashboard*) pasca-otentikasi yang menyajikan visualisasi ringkas mengenai kondisi vitalitas lahan. Dasbor ini mengintegrasikan widget metrik informatif untuk menampilkan akumulasi jumlah blok zona tanam dan ringkasan status aktivitas sensor telemetri secara real-time.

```

useEffect(() => {
  const verifiedCount = userList.filter(u => u.status === 'Verified').length;
  const onlineSensors = sensorStreams.filter(s => s && s.status === 'ONLINE').length;
  const offlineSensors = sensorStreams.filter(s => s && s.status === 'OFFLINE').length;

  setStats({
    totalPetani: userList.filter(u => u.role.toLowerCase() === 'petani' || u.role === 'USER').length,
    petaniTerverifikasi: verifiedCount,
    totalLahanAktif: 85,
    nodeSensorOnline: onlineSensors,
    nodeSensorOffline: offlineSensors,
    + const sensorStreams: SensorDataStream[]
  });
}, [usersList, sensorStreams]);

useEffect(() => {
  const interval = setInterval(() => {
    setSensorStreams((prevStreams) => {
      prevStreams.map((sensor) => {
        if (!sensor || sensor.status === 'OFFLINE') return sensor;
        return {
          ...sensor,
          kelembapanTanah: Math.max(30, Math.min(90, sensor.kelembapanTanah + (Math.random() > 0.5 ? 1 : -1))),
          suhuUdara: parseFloat((sensor.suhuUdara + (Math.random() > 0.5 ? 0.2 : -0.2)).toFixed(1)),
          lastUpdated: 'Baru saja'
        };
      });
    });
  }, 5000);
  return () => clearInterval(interval);
}, []);

const handleAddUser = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!newUserName || !newUserEmail) return;

```

- Implementasi klaster manajemen data operator pada antarmuka Content Management System (CMS). Modul halaman CMS dikembangkan khusus bagi administrator untuk memantau otorisasi akun pengguna. Komponen ini melakukan iterasi dinamis terhadap data yang ditarik dari fungsi findAllUsers backend, menyajikannya ke dalam baris tabel terstruktur yang memuat detail nama, email, dan hak akses operasional.

```

1 import React, { useState, useEffect } from 'react';
2 import { StyleSheet, Text, View, ScrollView, TouchableOpacity, useWindowDimensions, ActivityIndicator, Alert, Platform,
3 // Import Date Picker resmi Expo
4 import DateTimePicker from '@react-native-community/datetimepicker';
5 // [DITAMBAHKAN]: Import AsyncStorage untuk mekanisme persistence data lokal
6 import AsyncStorage from '@react-native-async-storage/async-storage';
7
8 interface ProfileProps {
9   userName: string; // Data nama dinamis dari user yang sedang login
10  userEmail: string; // Data email dinamis dari user yang sedang login
11  onLogout: () => void;
12  onBackToDashboard: () => void;
13 }
14
15 export default function ProfileScreen({ userName, userEmail, onLogout, onBackToDashboard }: ProfileProps) {
16   const { width } = useWindowDimensions();
17   const isDesktop = width > 768;
18
19   // [DIUBAH]: Default loading diubah menjadi true untuk menunggu pengecekan storage lokal selesai
20   const [loading, setLoading] = useState(true);
21
22   // Status Verifikasi Dinamis
23   const [isVerified, setIsVerified] = useState(false);
24   const [isModalVisible, setIsModalVisible] = useState(false);
25
26   // State Penampung Form Identitas
27   const [inputPhone, setInputPhone] = useState('');
28   const [birthDate, setBirthDate] = useState<Date>(new Date());
29   const [showDatePicker, setShowDatePicker] = useState(false);
30   const [isDateSelected, setIsDateSelected] = useState(false);
31
32   // STATE MANAJEMEN ALUR OTP (One-Time Password)
33   const [otpStep, setOtpStep] = useState<'INPUT_DATA' | 'INPUT_OTP'>('INPUT_DATA');
34   const [inputOtp, setInputOtp] = useState('');
35   const [generatedOtp, setGeneratedOtp] = useState(''); // Tempat menyimpan OTP simulasi
36   const [timer, setTimer] = useState(60); // Hitung mundur dalam detik
37

```

4. Penyusunan modul Halaman Profile untuk manajemen informasi personal pengelola. Halaman profil pengguna diintegrasikan untuk menampilkan detail kartu identitas digital (*operator digital card*) secara personal. Menggunakan skema ekstraksi data session dari token lokal, halaman ini merender informasi sensitif pengelola yang telah terisolasi dengan aman dari paparan kebocoran data di pohon DOM peramban.

```
// ● FUNGSI FETCH DATA ASLI DARI NESTJS (Dengan Auto-Sync Status Pompa)
const fetchRealtimeSensorData = async () => {
  try {
    let token = null;
    try { token = await SecureStore.getItemAsync('userToken'); } catch (e) { token = localStorage.getItem('userToken'); }

    const response = await fetch(API_URL, {
      method: 'GET',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': token ? `Bearer ${token}` : '',
      },
    });

    if (response.ok) {
      const result = await response.json();

      if (result) {
        // Sinkronisasi lampu indikator sakelar di HP secara otomatis dari state server
        if (result.statusPompa !== undefined) {
          setIsPumpActive(result.statusPompa);
        }
      }
    }
  }
}
```

5. Konfigurasi gerbang pengaman rute antarmuka (Protected Routes Guard) pada susunan struktur aplikasi. Demi mencegah akses ilegal langsung ke halaman internal (seperti melompati paksa ke halaman *overview* atau *CMS* tanpa login), diimplementasikan komponen proteksi rute di sisi klien. Komponen ini mengecek ketersediaan token di memori lokal; jika absen, rute otomatis memblokir rendering komponen dasbor dan melempar balik operator ke halaman login.

```

export default function LoginScreen({ navigation, onNavigateToRegister, initialEmail, onLoginSuccess }: LoginScreenProps) {
  const handleLogin = async () => {
    const response = await fetch(`${API_URL}/auth/login`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({ email, password }),
    });

    const result = await response.json();

    if (response.ok) {
      const token = result.access_token;
      if (token) {
        try {
          await SecureStore.setItemAsync('userToken', token);
        } catch (e) {
          localStorage.setItem('userToken', token);
          console.log('SecureStore dialihkan ke localStorage (Web Environment)');
        }
      }

      // Ekstraksi nama & email riil dari data respon API
      const fallbackName = email.split('@')[0];
      const userRealName = result.user?.name || result.name || fallbackName;
      const userRealEmail = result.user?.email || result.email || email;

      // 3. EKSEKUSI CALLBACK: Aman dan terikat parameter data profil secara dinamis
      if (typeof onLoginSuccess === 'function') {
        onLoginSuccess(userRealName, userRealEmail);
      }
    } else {
      Alert.alert('Login Gagal', result.message || 'Email atau password salah.');
```

6. Penerapan interceptor respons jaringan (Axios Global Interceptor) untuk sinkronisasi pemutusan sesi. Guna mengamankan sesi di seluruh halaman dasbor, ditambahkan logika interceptor yang memindai sinyal kesalahan dari server backend. Jika operator membuka halaman CMS atau Profile namun tokennya kedaluwarsa (HTTP 401 Unauthorized), frontend akan menangkap eksepsi tersebut secara otomatis, menghapus data lokal, dan mengarahkan paksa pengguna keluar kembali ke halaman login

